

# Automated File Upload Pipeline

From Local Folder → Amazon S3 → AWS Lambda → Amazon RDS → Amazon SNS

## Introduction

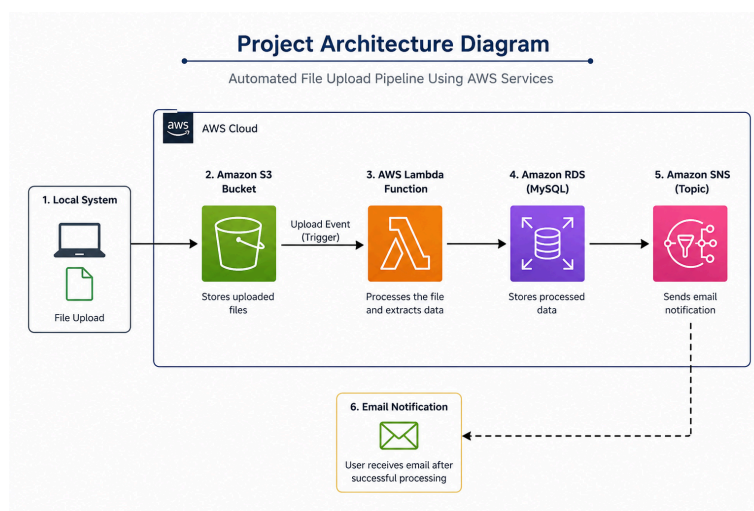
The **Automated File Upload Pipeline** is an AWS-based project that demonstrates how multiple cloud services work together to automate file processing. In this project, a file uploaded from a local system to an Amazon S3 bucket automatically triggers an AWS Lambda function. The Lambda function processes the file, stores the extracted data in an Amazon RDS database, and sends an email notification using Amazon SNS.

This project showcases an event-driven architecture and provides hands-on experience with Amazon S3, AWS Lambda, Amazon RDS, Amazon SNS, Git, and Visual Studio Code.

## Project Objectives

- Build an automated file upload pipeline using AWS.
- Upload files from a local system to Amazon S3.
- Trigger AWS Lambda automatically when a file is uploaded.
- Store processed data in Amazon RDS.
- Send email notifications using Amazon SNS.
- Understand and implement an event-driven AWS architecture.

## Project Architecture Diagram



## **Phase 1 – Project Setup**

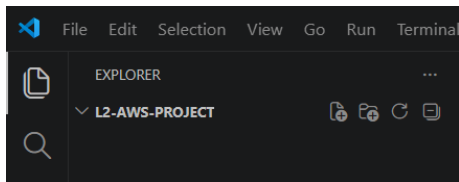
### **Objective**

Prepare the local development environment for the AWS L2 project by creating a dedicated project workspace. This workspace will be used to store the project files, documentation, screenshots, and source code throughout the implementation.

### **Step 1 – Create the Project Folder**

#### **Procedure**

- Created a new folder named **L2-AWS-Project**.
- Opened the project folder in **Visual Studio Code**.



**Figure 1: L2-AWS-Project folder created and opened in Visual Studio Code.**

#### **Verification**

- L2-AWS-Project folder created successfully.
- Project folder opened in Visual Studio Code.
- Workspace is ready for project development.

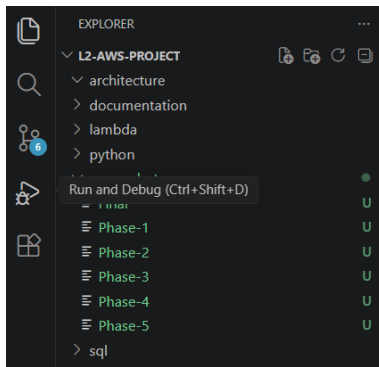
#### **Result**

The project folder was successfully created and opened in Visual Studio Code. The workspace is now ready for implementing the AWS L2 project.

### **Step 2 – Create the Project Structure**

#### **Procedure**

- Created the required project folders inside the **L2-AWS-Project** directory.
- Organized the **screenshots** folder into phase-wise subfolders for project documentation.



**Figure 2: Project folder structure created successfully in Visual Studio Code.**

## Verification

- Project folders created successfully.
- Screenshot folders organized phase-wise.
- Project structure is ready for implementation.

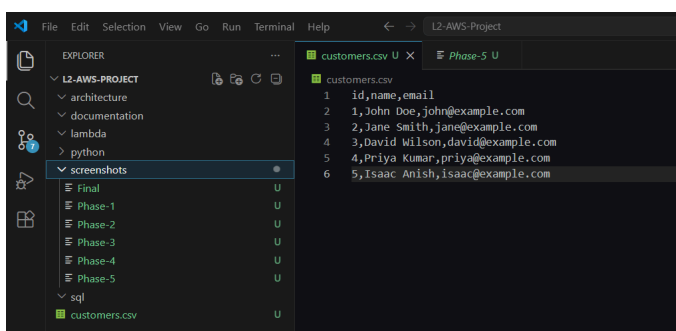
## Result

The project structure was successfully created and organized. The workspace is now ready for storing project files, documentation, source code, and screenshots.

## Step 3 – Create the Customer CSV File

### Procedure

- Created a CSV file named **customers.csv** in the project directory.
- Added sample customer records to be used for data processing in the AWS pipeline.



**Figure 3: Customer CSV file created successfully with sample customer records.**

## Verification

- **customers.csv** file created successfully.
- Sample customer records added in CSV format.
- File saved and ready for upload to Amazon S3.

## Result

The `customers.csv` file was successfully created with sample customer records. The file is now ready to be uploaded to the Amazon S3 bucket for further processing.

## Step 4 – Verify AWS Account and Select the AWS Region

### Procedure

- Logged in to the AWS Management Console using the Root User.
- Selected the Asia Pacific (Mumbai) – ap-south-1 Region for the project.

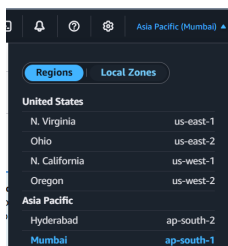


Figure 4: AWS Management Console configured with the Mumbai (ap-south-1) Region.

### Verification

- Root User login verified.
- Mumbai Region (ap-south-1) selected.
- AWS Console ready for resource creation.

## Result

The AWS account and project region were successfully verified. The environment is now ready for creating AWS resources.

## Phase 2 – Amazon S3 Configuration

### Objective

Configure Amazon S3 to securely store the `customers.csv` file, which will act as the input source for the automated file upload pipeline. The uploaded file will later trigger the AWS Lambda function for processing.

## Step 1 – Create the Amazon S3 Bucket

### Procedure

- Created an Amazon S3 bucket named **aws-l2-file-pipeline-2026** in the Asia Pacific (Mumbai) Region.
- Verified that the bucket was created successfully.

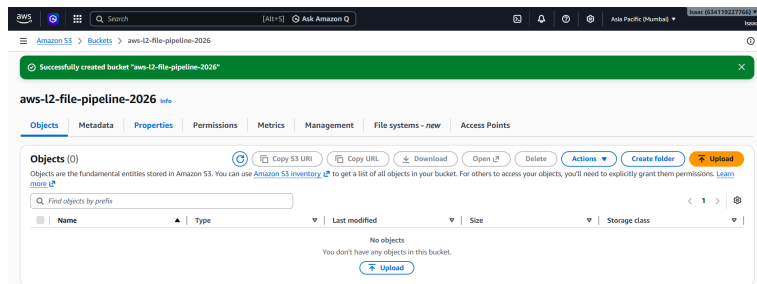


Figure 5: Amazon S3 bucket created successfully.

### Verification

- S3 bucket created successfully.
- Bucket opened without any errors.
- Bucket is ready to store project files.

### Result

The Amazon S3 bucket was successfully created and is ready to store the **customers.csv** file for the automated processing pipeline.

## Step 2 – Upload the Customer CSV File

### Procedure

- Uploaded the **customers.csv** file to the **aws-l2-file-pipeline-2026** Amazon S3 bucket.
- Verified that the file upload completed successfully without any errors.

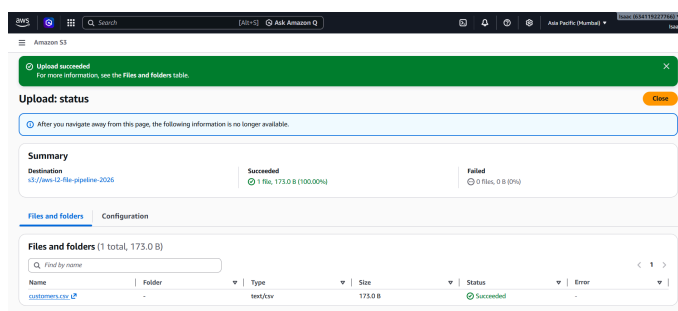


Figure 6: Customer CSV file uploaded successfully to the Amazon S3 bucket.

## Verification

- customers.csv uploaded successfully.
- Upload completed without errors.
- File stored in the Amazon S3 bucket.

## Result

The customer CSV file was successfully uploaded to the Amazon S3 bucket and is ready to be processed by the AWS Lambda function in the later stages of the project.

## Step 3 – Verify the Uploaded File

### Procedure

- Opened the Amazon S3 bucket.
- Verified that the uploaded **customers.csv** file was available in the bucket.

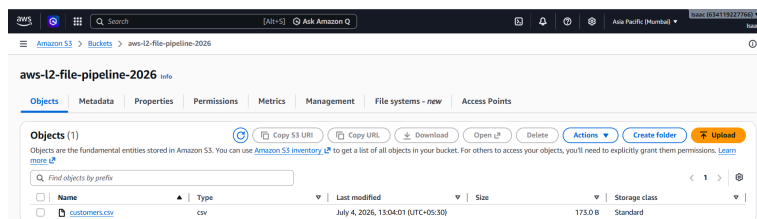


Figure 7: Customer CSV file verified successfully in the Amazon S3 bucket.

## Verification

- customers.csv is available in the bucket.
- File details are displayed correctly.
- File is ready for processing by AWS Lambda.

## Result

The uploaded customer CSV file was successfully verified in the Amazon S3 bucket and is ready for the next stage of the automated file upload pipeline.

## **Phase 3 – Amazon SNS Configuration**

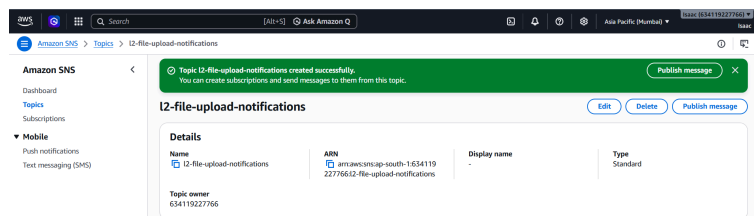
### **Objective**

Configure Amazon Simple Notification Service (SNS) to send email notifications whenever the AWS Lambda function processes the uploaded **customers.csv** file. This ensures that the user receives an email indicating whether the file was processed successfully or if an error occurred.

### **Step 1 – Create an Amazon SNS Topic**

#### **Procedure**

- Opened the Amazon SNS service and created a Standard topic.
- Configured the topic name as l2-file-upload-notifications and created the topic.



**Figure 8: Amazon SNS topic created successfully.**

#### **Verification**

- SNS topic created successfully.
- Topic ARN generated.
- Topic is ready for creating subscriptions.

#### **Result**

The Amazon SNS topic was successfully created and is ready to send email notifications to subscribed users.

### **Step 2 – Create and Confirm the Email Subscription**

#### **Procedure**

- Created an email subscription for the SNS topic.
- Confirmed the subscription using the verification email sent by Amazon SNS.

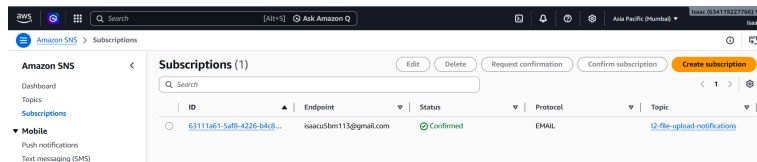


Figure 9: Amazon SNS email subscription created and confirmed successfully.

## Verification

- Email subscription created successfully.
- Subscription status is **Confirmed**.
- SNS topic is ready to send email notifications.

## Result

The email subscription was successfully created and confirmed. Amazon SNS is now configured to send notification emails for the project.

## Phase 4 – Amazon RDS (MySQL) Configuration

### Objective

Create and configure an **Amazon RDS MySQL** database instance to securely store the customer records processed by the AWS Lambda function. This database will receive the data extracted from the uploaded **customers.csv** file.

### Step 1 – Create the RDS Security Group

#### Procedure

- Created a security group named **l2-rds-sg**.
- Configured the MySQL/Aurora inbound rule on port **3306**.
- Used the default outbound rule.

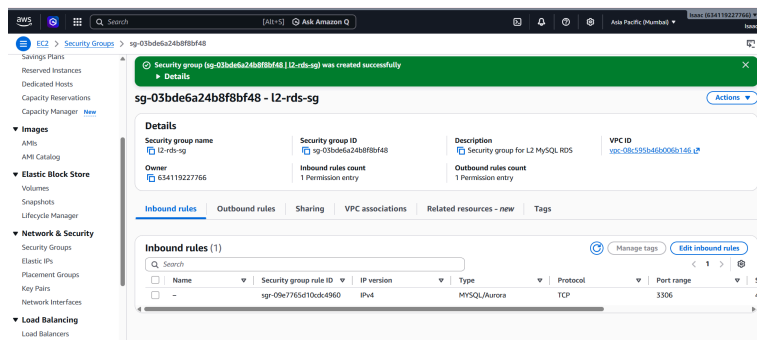


Figure 10: Security group created successfully for the Amazon RDS MySQL instance.



## Verification

- Security group created successfully.
- MySQL inbound rule configured.
- Security group ready for Amazon RDS.

## Result

The security group was successfully created and configured for the Amazon RDS MySQL instance.

## Step 2 – Create the Amazon RDS MySQL Instance

### Procedure

- Created the Amazon RDS MySQL database instance.
- Waited until the database status changed to **Available**.

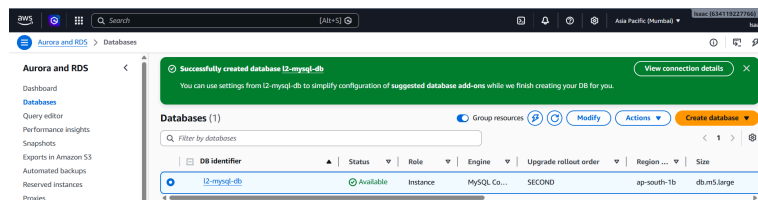


Figure 11: Amazon RDS MySQL database created successfully with Available status.

## Verification

- Database instance created successfully.
- Status is **Available**.
- Database is ready for connectivity.

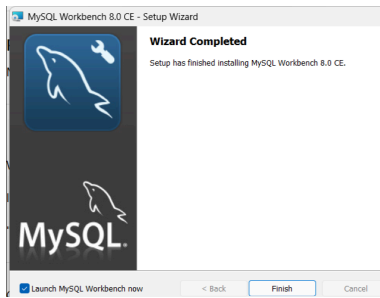
## Result

The Amazon RDS MySQL database instance was successfully created and is ready for use.

## Step 3 – Install MySQL Workbench

### Procedure

- Downloaded MySQL Workbench from the official MySQL website.
- Installed the application using the default settings.
- Completed the installation successfully.



 **Figure 12: MySQL Workbench installed successfully.**

## Verification

- Installation completed successfully.
- MySQL Workbench is ready to launch.

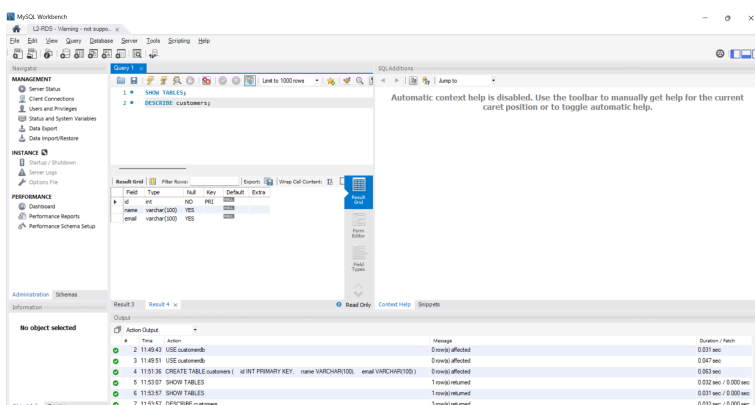
## Result

MySQL Workbench was successfully installed and is ready to connect to the Amazon RDS database.

## Step 4 – Create and Verify the Customers Table

### Procedure

- Connected to the Amazon RDS MySQL database using MySQL Workbench.
- Selected the `customerdb` database using the `USE customerdb;` command.
- Created the `customers` table with the required columns (`id`, `name`, and `email`) using the `CREATE TABLE` statement.
- Verified the table creation using the `SHOW TABLES;` command.
- Checked the table structure using the `DESCRIBE customers;` command.



 **Figure 13: Customers table created and verified successfully in MySQL Workbench.**

## Verification

- ✓ The **customers** table was created successfully in the **customerdb** database.
- ✓ The **SHOW TABLES;** command confirmed that the table exists.
- ✓ The **DESCRIBE customers;** command verified the table structure with the required columns.

## Result

The **customers** table was successfully created and verified in the Amazon RDS MySQL database. The database is now ready to store customer records and proceed with the AWS Lambda automation phase.

## Phase 5 – AWS Lambda Automation

### Objective

Create an AWS Lambda function that automatically:

- Reads the CSV file uploaded to the S3 bucket.
- Connects to the Amazon RDS MySQL database.
- Inserts the CSV records into the **customers** table.
- Sends an email notification using Amazon SNS.

### Step 1 – Create IAM Role for Lambda

#### Procedure

- Opened the IAM service and created a new role.
- Selected Lambda as the trusted service.
- Attached the required AWS managed policies.
- Created the role **I2-lambda-role** successfully.

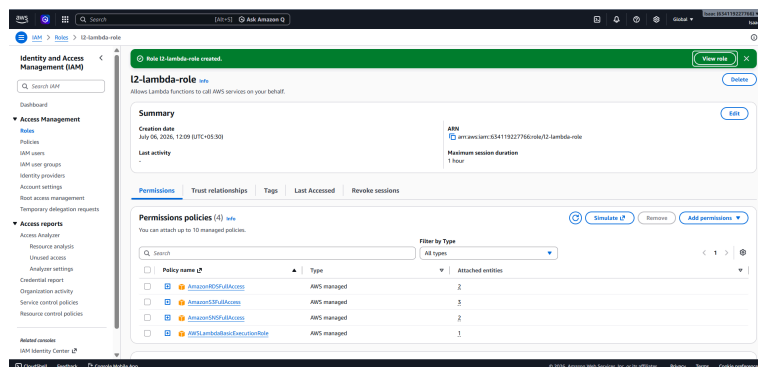


Figure 14: IAM role (I2-lambda-role) created successfully with the required permissions.

## Verification

- ✓ IAM role created successfully.
- ✓ Required policies attached to the role.
- ✓ Trusted entity configured as AWS Lambda.

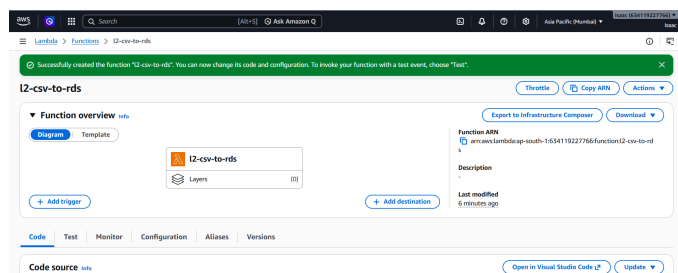
## Result

The IAM role was successfully created with the required permissions for AWS Lambda to access Amazon S3, Amazon RDS, Amazon SNS, and CloudWatch Logs.

## Step 2 – Create AWS Lambda Function

### Procedure

- Opened the AWS Lambda service.
- Created a new Lambda function using **Author from scratch**.
- Configured the function with the required runtime and IAM role.
- Created the Lambda function successfully.



 **Figure 15: AWS Lambda function (l2-csv-to-rds) created successfully.**

## Verification

- Lambda function created successfully.
- Function name configured as **l2-csv-to-rds**.
- IAM execution role attached to the function.

## Result

The AWS Lambda function was successfully created and is ready for uploading the Python code to process CSV files from Amazon S3 and insert the data into the Amazon RDS MySQL database.

## Step 3 – Develop the Lambda Function

### Procedure

- Opened the AWS Lambda function **l2-csv-to-rds**.
- Created the **lambda\_function.py** file.
- Imported the required Python libraries (**boto3**, **csv**, **json**, **os**, and **pymysql**).
- Implemented the logic to read the uploaded CSV file from Amazon S3.
- Configured the Lambda function to connect to the Amazon RDS MySQL database.
- Added Amazon SNS notification logic to send email notifications for successful and failed executions.
- Saved the Lambda function code.

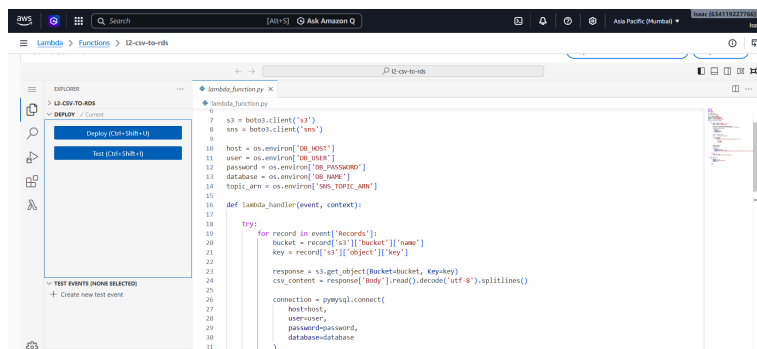


Figure 16: Lambda function code created successfully.

### Verification

- Lambda function code created successfully.
- Required Python libraries imported successfully.
- Amazon S3 file processing logic implemented.
- Amazon RDS database connection configured.
- Amazon SNS notification logic added successfully.
- Code saved without syntax errors.

### Result

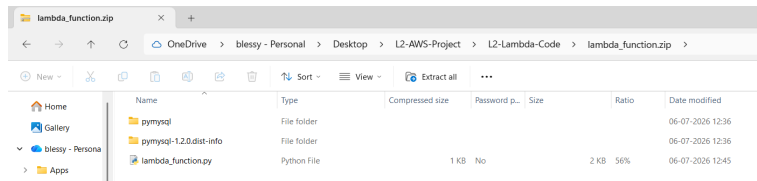
The Lambda function was successfully developed to process uploaded CSV files from Amazon S3, insert customer records into the Amazon RDS MySQL database, and send Amazon SNS email notifications indicating successful or failed processing.

## Step 4 – Create the Lambda Deployment Package

### Procedure

- Created a deployment package containing the Lambda function code and required dependencies.

- Selected `lambda_function.py`, `pymysql`, and `pymysql-1.2.0.dist-info`.
- Compressed the selected files into `lambda_function.zip`.



**Figure 17: Lambda deployment package (`lambda_function.zip`) created successfully.**

## Verification

- Deployment package created successfully.
- ZIP file contains `lambda_function.py`.
- Required PyMySQL library included in the package.

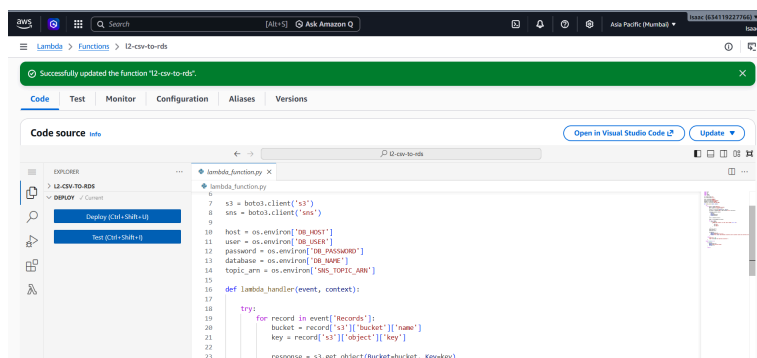
## Result

The Lambda deployment package was successfully created and is ready to be uploaded to the AWS Lambda function.

## Step 5 – Upload the Lambda Deployment Package

### Procedure

- Opened the AWS Lambda function `l2-csv-to-rds`.
- Uploaded the `lambda_function.zip` deployment package.
- Saved and deployed the updated Lambda function code.



**Figure 18: Lambda deployment package uploaded successfully.**

## Verification

- Deployment package uploaded successfully.
- Lambda function updated without errors.
- Python code and required PyMySQL dependency are visible in the Code Source section.

## Result

The Lambda deployment package was successfully uploaded to the AWS Lambda function and is ready for configuration.

## Step 6 – Configure Lambda Environment Variables

### Procedure

- Opened the **Lambda** function **l2-csv-to-rds**.
- Navigated to **Configuration** → **Environment variables**.
- Added the required database and SNS environment variables.
- Saved the configuration successfully.

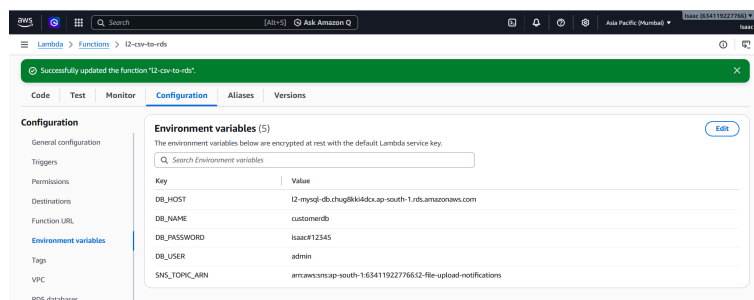


Figure 19: Lambda environment variables configured successfully.

### Verification

- Environment variables added successfully.
- Database connection details configured.
- SNS Topic ARN configured for email notifications.
- Configuration saved without errors.

## Result

The Lambda function was successfully configured with the required environment variables, enabling it to connect to the Amazon RDS database and send SNS email notifications after processing uploaded CSV files.

## Step 7 – Configure S3 Event Trigger

### Procedure

- Opened the Lambda function **l2-csv-to-rds**.

- Clicked **Add Trigger** and selected **Amazon S3**.
- Selected the project S3 bucket and configured the object creation event.
- Added the S3 trigger to the Lambda function.

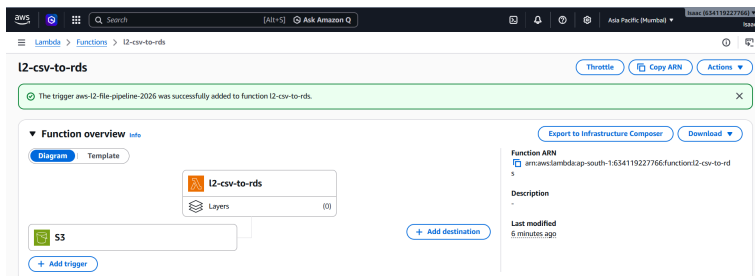


Figure 20: Amazon S3 trigger configured successfully for the Lambda function.

## Verification

- S3 bucket trigger added successfully.
- Lambda function linked with the S3 bucket.
- Object creation event configured successfully.

## Result

The Lambda function is successfully configured to automatically execute whenever a CSV file is uploaded to the Amazon S3 bucket.

## Step 8 – Upload CSV File to Amazon S3

### Procedure

- Opened the Amazon S3 bucket.
- Deleted the existing **customers.csv** file.
- Uploaded a fresh **customers.csv** file to the S3 bucket.
- Waited for the upload to complete successfully.

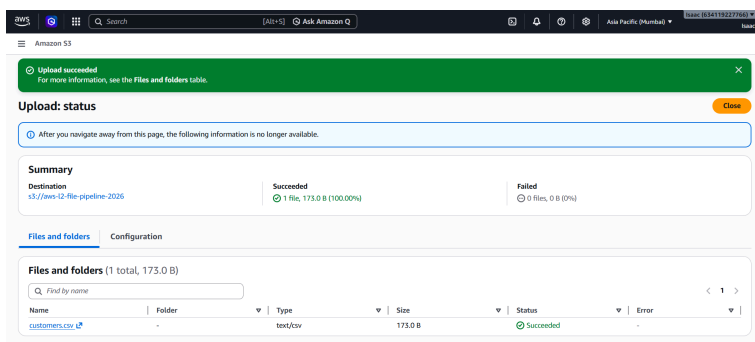


Figure 21: customers.csv uploaded successfully to the Amazon S3 bucket.



## Verification

- CSV file uploaded successfully.
- Upload status displayed as **Succeeded**.
- S3 ObjectCreated event generated.

## Result

The CSV file was successfully uploaded to the Amazon S3 bucket, automatically triggering the Lambda function for further processing.

## Step 9 – Verify Lambda Execution

### Procedure

- Opened the Lambda function **l2-csv-to-rds**.
- Navigated to the **Monitor** tab.
- Opened the latest **CloudWatch Log Stream**.
- Verified the Lambda execution logs.

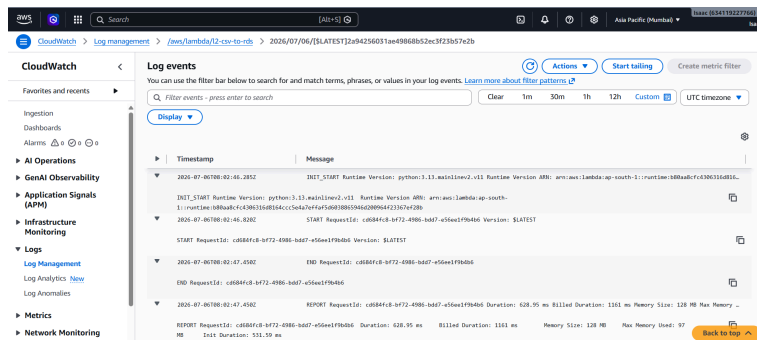


Figure 22: CloudWatch Logs showing successful Lambda execution.

## Verification

- Lambda invocation started successfully.
- Execution completed without errors.
- CloudWatch generated execution logs for the function.
- CSV file processed successfully.

## Result

The Lambda function executed successfully after the CSV file was uploaded to the Amazon S3 bucket.

## Step 10 – Verify Data in Amazon RDS

### Procedure

- Opened MySQL Workbench and connected to the Amazon RDS database.
- Selected the **customerdb** database.
- **SELECT \* FROM customers;**
- Verified the inserted records in the **customers** table.
- Verified the Amazon SNS email notification confirming successful data insertion.

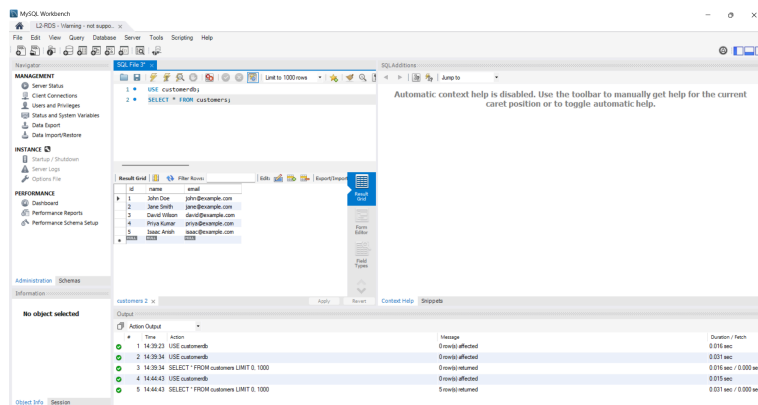


Figure 23: Customer records successfully inserted into the Amazon RDS database.

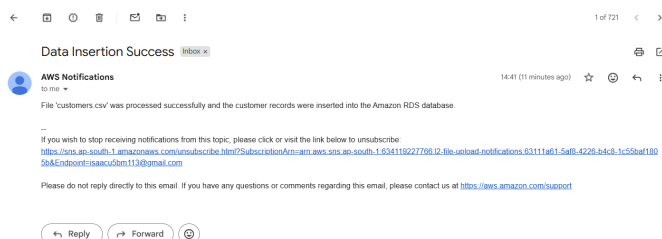


Figure 24: Amazon SNS email notification confirming successful data insertion.

### Verification

- Records retrieved successfully from the customers table.
- Customer details matched the uploaded CSV file.
- Amazon SNS success email received.
- Data insertion through Lambda verified successfully.

### Result

The uploaded CSV file was successfully processed by the AWS Lambda function, customer records were inserted into the Amazon RDS MySQL database, and a success notification was delivered through Amazon SNS.

## **Phase 6 – Documentation Completion**

### **Step 1: Project Conclusion**

#### **Conclusion**

The AWS L2 CSV-to-RDS Automation Project was successfully implemented using multiple AWS services, including Amazon S3, AWS Lambda, Amazon RDS MySQL, Amazon SNS, IAM, and CloudWatch. The solution automatically processes customer data uploaded as CSV files to an S3 bucket, inserts the records into an Amazon RDS MySQL database, and sends email notifications through Amazon SNS upon successful or failed execution.

The project also demonstrated event-driven serverless architecture by integrating Amazon S3 event notifications with AWS Lambda. CloudWatch Logs were used to monitor Lambda execution, and MySQL Workbench was used to verify that the customer records were successfully inserted into the database. Overall, the project successfully automated the complete CSV processing workflow while improving scalability, reliability, and operational efficiency.

### **Step 2: Challenges Faced**

#### **Challenges Faced**

During the implementation of the AWS L2 Project, several technical challenges were encountered and resolved:

- Initially, the Lambda deployment package failed because the required `pymysql` library was not included.
- Lambda execution failed until the correct database environment variables were configured.
- Amazon SNS email notifications were not received until the subscription was confirmed.
- Uploading the same CSV file multiple times resulted in a duplicate primary key error because the customer records already existed in the database.
- While clearing the customer table, MySQL Workbench Safe Update Mode prevented the DELETE command from executing.
- A temporary database connection timeout occurred while attempting to truncate the table, which was resolved by reconnecting to the Amazon RDS instance.
- CloudWatch Logs helped identify and troubleshoot Lambda execution errors during testing.

## Step 3: Learning Outcomes

### Learning Outcomes

This project helped in gaining practical experience with AWS serverless services and cloud database integration. The following concepts were learned:

- Creating and configuring Amazon S3 buckets.
- Developing AWS Lambda functions using Python.
- Creating Lambda deployment packages with external Python libraries.
- Configuring Lambda environment variables.
- Connecting AWS Lambda with Amazon RDS MySQL.
- Creating and managing IAM roles and permissions.
- Configuring Amazon SNS for email notifications.
- Monitoring Lambda execution using Amazon CloudWatch Logs.
- Automating file processing using Amazon S3 event triggers.
- Verifying data using MySQL Workbench.

## Step 4: Cleanup Steps

### Cleanup Steps

After successful completion of the project, the following AWS resources can be deleted to avoid unnecessary charges:

- Delete the Amazon S3 bucket and all uploaded objects.
- Delete the AWS Lambda function.
- Delete the Amazon RDS MySQL database instance.
- Delete the Amazon SNS topic and subscription.
- Delete IAM roles created specifically for the project (if not reused).
- Delete CloudWatch Log Groups associated with the Lambda function.
- Remove any unused security groups created for the project.

## Step 5: Architecture Summary

### Architecture Summary

**The project follows an event-driven serverless architecture.**

1. A CSV file is uploaded to the Amazon S3 bucket.
2. Amazon S3 generates an ObjectCreated event.
3. AWS Lambda is automatically invoked.
4. The Lambda function reads the uploaded CSV file.

5. Lambda connects securely to the Amazon RDS MySQL database.
6. Customer records are inserted into the **customers** table.
7. Amazon SNS sends a success or failure email notification.
8. Amazon CloudWatch Logs record the execution details for monitoring and troubleshooting.

## **AWS L2 Project**

### **Automated File Upload Pipeline**

#### **Submitted By:**

Isaac Anish

**Thank You**

